

Fullstack Engineer Challenge

Goal

Your goal for this exercise is to dive into a PRD, understand what needs to be built, and design the system that would attend the requirements. The PRD itself will contain some loose requirements and some ambiguity too, but please feel free to make assumptions to solve it and please share with us why you assume those parameters so we can better understand your decision making process!

Rules

Time limit: 48 hours

Please don't use AI to solve the assignment itself, but we using it to debate possibilities and research is more than welcome

Context

Alex, a Product Manager at a media company, sent you this PRD and system diagram for a new video platform. Alex was a developer years ago but has been in business roles since. They recently took some online courses on video streaming and tried their best to document the requirements and architecture based on research and AWS webinars.

PRD (page 1)

Summary

Users upload short videos (≤ 10 min, ≤ 1 GB). The system creates a few quality versions (e.g., 360p/720p/1080p), generates a poster image and thumbnail strip, and serves playback with a simple web player. Candidates will review an intentionally flawed design for this PRD and propose improvements.

In Scope

- Web upload with local progress, cancel, and retry of transient errors.
- Server-side processing: basic virus scan, metadata extraction (duration, dimensions), transcode into 2–3 MP4 quality versions, poster, and thumbnail sprites.
- Playback page with a custom player using standard HTML5 video:
 - Play/pause, seek, volume/mute.
 - Streaming;
 - Manual quality selector (user can switch 360p/720p/1080p).
 - Optional captions upload/display (caption file format: .vtt, optional).
- Delivery via CDN (cache friendly, URLs should be versioned/immutable).

Out of Scope

- Recommendations, social feed, DRM, downloads/offline.
- Cross-device progress sharing (not required in this version).

PRD (page 2)

Functional Requirements

Upload

- Accept MP4 (H.264/AAC), MOV, or WebM input.
- After upload completes, return an ID the client can poll for processing status.

Processing

- Produce multiple quality versions (exact bitrates/resolutions are up to the team).
- Store originals and derived files in object storage.
- Create poster image and a thumbnail sprite (for trick-play preview).
- Attach optional captions to the video record.

Playback

- Fetch a playback manifest JSON describing available files and metadata (no streaming protocol specifics required).

Delete

- A deleted video should make playback links unusable;
- Deletion propagation: player should fail within ≤ 60 s of delete.

The player starts with a reasonable default (e.g., 720p) and lets the user switch quality (reloading the source is acceptable).

Access & Moderation

- Deleting a video should make playback links unusable quickly.

Non-Functional Targets

- Upload $\rightarrow$ first playable file ready: ≤ 15 s P95 for a short “preview” version.
- Player start time (broadband ≥ 10 Mbps): ≤ 2.0 s P95.
- Availability: Playback path 99.9%, Upload/processing APIs 99.0% monthly.
- Scale assumptions: up to 500 concurrent uploads, 20k uploads/day, 5k concurrent viewers.

Constraints & Assumptions

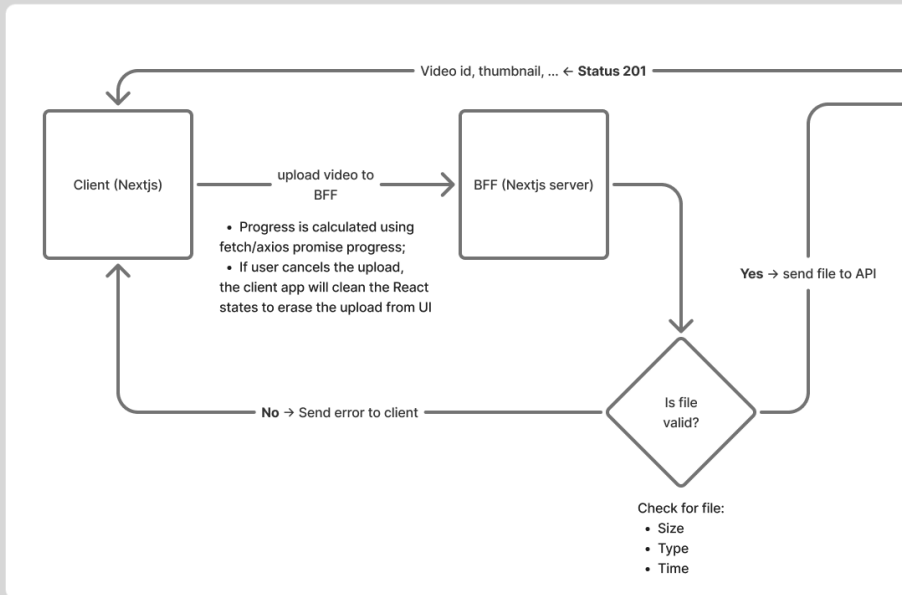
- Object storage + CDN; files are immutable versioned URLs (cache-friendly).
- No requirement for adaptive streaming protocols; multiple MP4 qualities are sufficient.
- Captions optional; if provided, they’re served as static files.

Success Metrics

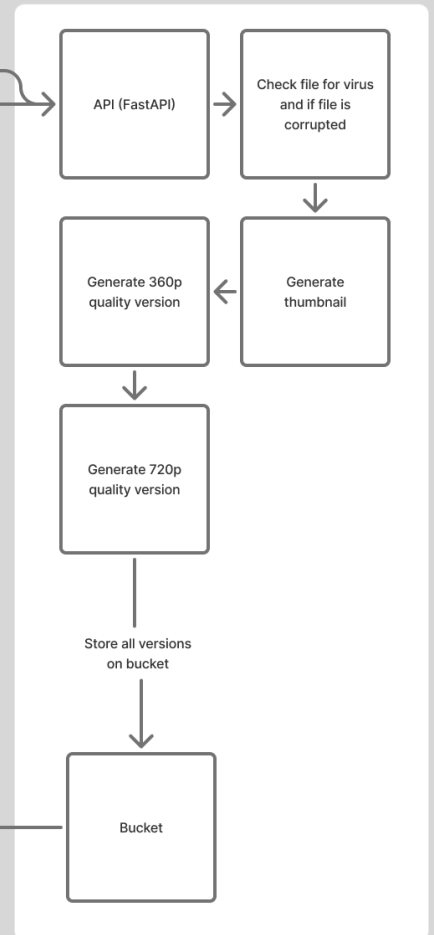
- Upload-to-play success rate $\geq 95\%$ within 30 minutes.
- P95 processing time for a 10-minute source ≤ 10 minutes.
- Playback start abandonment $\leq 5\%$.

System diagram

Upload



Processing



Playback

